

프로젝트 계획서

월 식비 예산 기반 밀플래닝 서비스 · mealplanning (mp)

팀 이름	mealplanning (프로젝트 접두사 mp)
구성원	· 봉수 Project Leader : 프로젝트 총괄 · AWS 이관 설계 · Terraform/Ansible IaC · 클러스터 운영 · 태현 Infra / Platform : Kubernetes 플랫폼 · 노드/스토리지 · 건우 Dev (App) : 애플리케이션 개발 · CI/CD 파이프라인 · AI 서버리스 트랙 · 정현 Observability : 관측(Prometheus-Loki-Tempo-Grafana) · 알림 · FinOps 대시보드 · 정은 Data : 데이터 파이프라인 · 수집 · PostgreSQL/Elasticsearch 스키마
프로젝트 주제	월 식비 예산을 기준으로 식단을 계획하고 실제 장바구니 비용까지 추적하는 밀플래닝 서비스. 레시피에서 재료를 자동 추출하고 온라인 마켓의 현재가와 대조하여, 해당 레시피의 실제 조리 비용을 산출한다.
프로젝트 목표	■ 문제 정의 기존 레시피 서비스는 조리 방법만, 가격 비교 서비스는 최저가 정보만 제공한다. 정해진 월 예산 안에서 식단을 결정하는 단계를 지원하는 서비스는 부재한 상황이다. ■ 목표 ① 레시피 → 재료 → 현재가 → 총 재료비까지 하나의 흐름으로 잇는다. ② 월 예산을 입력하면 그 안에 들어오는 식단을 제안하고, 지출을 캘린더로 추적한다. ③ 냉장고 재고와 영수증 내역을 반영하여 보유 재료를 제외한 금액을 산출한다. ④ 상기 기능을 상용 GPU 및 상용 LLM 의존 없이 CPU 기반 자체 모델로 구현한다. ■ 인프라 목표 기능 확장성과 부하 확장성 확보를 위해 MSA 로 설계하고, 온프레미스 Kubernetes 로 구축한 뒤 AWS EKS 로 이전한다. 이전 완료 후 두 사이트는 DR 관계로 구성한다.

기술 스택

1. IaaS	Amazon EC2 (m7g.xlarge Graviton · t4g.xlarge · t3.medium) · Amazon EBS (gp3) · Amazon VPC NAT Gateway · Internet Gateway · VPC Endpoint (S3 Gateway · SQS/SecretsManager/STS Interface) (온프레미스) Proxmox VE · 물리 서버 2 대 · VirtualBox
2. PaaS	■ 오케스트레이션 Amazon EKS 1.34 (Graviton/arm64) · Kubernetes 1.34.10 (kubeadm · 온프레임) · Docker · containerd 2.2.6 ■ 네트워크 · 메시 Cilium 1.19.6 (eBPF · kube-proxy 대체) · Istio 1.30.3 (sidecar · mTLS STRICT) · Gateway API ALB · ACM · AWS WAF · MetalLB (온프레임) · Cloudflare Tunnel ■ 데이터

	CloudNativePG (PostgreSQL) · ECK (Elasticsearch + nori) · Amazon ElastiCache for Valkey Strimzi Kafka (온프레임) · Amazon S3 · Amazon SQS · PgBouncer · PGSync (CDC) ■ 배포 · 자동화 Terraform · Ansible · Argo CD · Argo Rollouts (Blue-Green) · Kustomize · Helm GitLab CI · Jenkins · Amazon ECR · Harbor · SonarQube · Trivy ■ 스케일 HPA · KEDA (SQS/Kafka 이벤트 기반 scale-to-zero) · Karpenter (노드) ■ 보안 AWS IAM (IRSA) · AWS Secrets Manager · External Secrets Operator · cert-manager Amazon GuardDuty · AWS CloudTrail · NetworkPolicy · CiliumNetworkPolicy · Pod Security Admission ■ AI AWS Lambda (함수 10 종) · Amazon Bedrock · Amazon API Gateway ■ 관측 Prometheus (kube-prometheus-stack) · Loki · Tempo · Grafana · Alloy · Amazon CloudWatch · kubecost
3. SaaS	GitHub (초기 코드 정본 · Issues · PR 리뷰) → GitLab (2026-08-14 코드 정본 전환 · MR · CI) Slack (알림 채널 #mp-alerts · #mp-security · 일일 운영 리포트) Cloudflare (DNS · Tunnel · WAF IP allowlist) Google Gemini API (영수증 OCR · 영상 재료 추출) · YouTube Data API ■ 문서 관리 — 별도 SaaS 없이 코드와 동일 저장소의 Markdown 으로 관리 (Docs as Code · 143 개 파일 43,683 줄) 인프라 변경 PR 은 문서 갱신을 포함하지 않을 경우 리뷰 단계에서 반려하여, 코드와 문서의 정합성을 유지한다.
4. Framework	■ 백엔드 — FastAPI (Python 3.12) · psycopg3 (raw SQL · ORM 미사용) · uvicorn ■ 프론트엔드 — React + Vite + TypeScript (PWA) · nginx 정적 서빙 ■ AI/ML — CRF (한식 재료 NER) · XGBoost (신선도 예측) · LightGBM (레시피 랭킹) · scikit-learn ■ 수집 — Playwright (Chromium) · BeautifulSoup ■ 테스트 — pytest (DB-free 테스트 · AppCtx 주입 seam)

서비스 흐름도 (아키텍처)

전체 구성	사용자 → Cloudflare(DNS) → ALB (ACM TLS 종단 · AWS WAF) → Istio Gateway (ns mp-ingress) → 애플리케이션 12 종 (ns app · Istio 사이드카 · mTLS STRICT) → 데이터 티어 (ns data — CloudNativePG · Elasticsearch/ECK) + Amazon ElastiCache for Valkey → 수집 파이프라인 (ns pipeline — SQS 컨슈머 · CronJob) 온프레미스(DR 대기 + 크롤 상시 프로덕션) — 크롤 CronJob 7 종 → Kafka → 릴레이 → S3 → 이벤트 알림 → SQS → 컨슈머 → PostgreSQL
-------	---

사이트 구성	■ AWS (ap-northeast-2) — 프로덕션 목표 사이트 VPC 2 개 : mp-vpc-service (10.10.0.0/16) · mp-vpc-ci (10.11.0.0/16 · 피어링 없음) AZ 2 개 (a · b) · 공개/노드/데이터 3 티어 서브넷 · EKS 노드 m7g.xlarge × 2~4 ■ 온프레미스 (Proxmox 물리 2 대 + VirtualBox 1 대) kubeadm 5 노드 (control plane 1 + worker 4) · MetalLB · Harbor · Jenkins 이관 후 역할 = ① DR 대기(Warm Standby) + ② 크롤 상시 프로덕션 (이중역할) ■ 두 사이트 연결 = S3 WAL 아카이브 경유 PostgreSQL 복제 (RPO 5 분 · 상시 터널 없음)
도면	docs/design/k8s-onprem-topology-ref.svg — 온프레미스 클러스터 토폴로지 docs/design/msa-monolith-to-microservices.svg — MSA 전환 개념도 docs/design/aws-one-host-to-three-az.svg — 단일 호스트 → 다중 AZ docs/mp-aws-topology.html — AWS 전체 토폴로지 (정본) docs/mp-aws-flow-diagrams.html — 요청-데이터 흐름

워크플로우

EKS 환경 (컨테이너)	① 개발자가 GitLab 에 push / MR 생성 ② GitLab CI — 변경 감지 → pytest 게이트 → SonarQube(측정) → docker build(arm64) ③ Trivy 취약점 게이트 (CRITICAL 차단) → Amazon ECR push (:sha · :X.Y.Z · :latest 3 태그) ④ CI 가 config 레포의 overlays/eks 이미지 태그를 :sha 로 커밋 ← CD 인계 지점 ⑤ Argo CD 가 config 레포를 감지 → 클러스터에 반영 (push → 약 2 초) ⑥ account · recipe 는 Argo Rollouts Blue-Green 으로 배포 (preview 기동 → AnalysisRun 통과 → 승격) ⑦ 부하 증가 시 HPA 가 파드를, Karpenter 가 노드를 증설 (노드 Running 도달 51 초 · 실측) ※ 매니페스트 이미지 태그는 :sha 로 고정한다. :latest 사용 시 Argo CD 가 변경을 감지하지 못하여 롤백이 불가능하다.
서버리스 환경 (AI)	① 사용자 요청 → Amazon API Gateway → AWS Lambda (함수 10 종) ② Lambda → Amazon Bedrock (생성) / PostgreSQL (RAG 근거 조회) ③ 배치 성격 작업은 EventBridge 스케줄 → Lambda → 결과 적재 ④ 비밀은 AWS Secrets Manager 에서 주입 (VPC Endpoint 경유 · NAT 폴백 없음) ※ EKS 애플리케이션 이전과는 별개의 독립 트랙으로 설계하였다. IAM 그룹 mealplanning-ai 로 권한 경계를 분리하고, mp-ai-* 명명 규칙을 벗어난 리소스는 접근을 거부한다.
수집 파이프라인	① 온프레미스 CronJob 7 종 (KST) — 마켓컬리 · 오아시스마켓 · 만개의레시피 수집 ② 수집 원본 → Kafka (retail.crawl.raw · retail.deal.raw · recipe.crawl.raw) ③ 릴레이가 Kafka → Amazon S3 로 운반 ④ S3 버킷 이벤트 알림 → Amazon SQS → KEDA 가 컨슈머를 0 에서 기동 ⑤ 컨슈머가 정제 후 PostgreSQL 적재 → PGSync CDC → Elasticsearch 색인 ※ 크롤 실행 사이트는 AWS 가 아닌 온프레미스이며, 이전 완료 후에도 온프레미스에서 상시 운영한다.
협업 · 품질	■ 코드 — 앱 레포는 PR/MR 리뷰 필수 (직접 머지 금지) · config 레포는 직접 머지 허용 ■ 이슈 — GitHub Issues → GitLab Issues (라벨 트리아지 5 종)

	■ 문서 — 영역별 정본(SSOT) 문서를 단일화하고 재파생 금지 원칙을 명시 ■ 알림 — Slack #mp-alerts (Alertmanager) · #mp-security (GuardDuty·CloudTrail 직결) ※ 보안 알림은 Alertmanager 를 경유하지 않는다. 관측 스택 장애 시 보안 알림도 함께 중단되는 순환 의존을 배제하기 위함이다.
--	--

주요 기능

1. 레시피 검색 · 추천	· 만개의레시피 기반 레시피 DB 를 Elasticsearch(한국어 nori 형태소 분석)로 색인해 검색 · 조리시간 · 난이도 · 인분 필터 · 썸네일 카드 제공 · LightGBM 랭킹 모델이 사용자 이력 기반으로 정렬 (CPU 서빙)
2. 재료 자동 추출 (AI)	· 레시피 본문에서 한식 재료를 CRF NER 로 추출 · Gazetteer 4 단계 매칭(정확 → 최장 접미 → 최장 토큰 → 최장 prefix)으로 표준 품목에 정규화 · 레시피 재료 91,164 건의 핵심어 빈도에서 표준 품목 약 300 종을 도출
3. 실시간 최저가 비교	·마켓컬리 · 오아시스마켓의 신선/가공 상품 현재가를 정기 폴링 + 롱테일 온디맨드로 수집 · 레시피 재료를 상품에 매칭하여 레시피별 총 재료비를 산출 (용량 × 단가 · 인분 반영) · 개수 → 무게 환산으로 정확도 보정
4. 월 예산 기반 식단 계획	· 월 식비 예산을 입력하면 그 안에 들어오는 식단을 구성 · 캘린더에 지출을 기록하고 예산 대비 소진율을 추적 · 자연어 질의(예: '3 만원으로 뭐 해먹지')를 챗봇이 예산 필터로 변환
5. 냉장고 재고 · 영수증 OCR	· 영수증 사진을 Gemini 로 인식해 품목·수량·금액을 구조화 (HITL 확정 저장) · 확정된 품목이 냉장고 재고로 들어가 식단 계산에서 제외 · 소비기한 임박 품목 우선 소비 유도
6. 최저가 알림	· 30 일 윈도우 통계 이상탐지($Z\text{-score} \leq -2.0$ · 성숙 표본 28 건 이상)로 비정상 저가를 탐지 · 관심 품목 등록자에게 fan-out 알림 발송 (다자간 트래픽 요구사항 반영)
7. AI 챗봇 (밥풀이)	· 의도분류 + 템플릿 + RAG. 레시피 · 가격 · 영양 · 예산 · 기능 안내를 한 창에서 처리 · 멀티턴 맥락(Redis 단기세션) · 세션 스코프 개인화('돼지고기 빼줘') · 출력 근거 대조로 재료 치환·생성 환각을 차단 · 오프토픽 관련성 게이트 적용
8. 영상 URL → 레시피 추출	· 사용자가 YouTube URL 을 넣으면 YouTube Data API + Gemini 멀티모달로 재료를 추출 · 추출 결과를 CRF NER 로 표준 품목에 정규화 · 외부 유료 API 를 사용하는 유일한 기능 (예외 승인 건)
9. 레시피북 · 공유	· 개인 레시피북 저장 · 공유 링크 발행 · 영양소 분석은 식약처 표준 데이터 DB 룩업 (AI 아님)

사용 기술 — 선택 근거

MSA 채택 근거	■ 기능 확장성 — 레시피 검색에서 출발하여 가격 비교 · 냉장고 · 영수증 인식 · 찻봇까지 추가되어 현재 12 중이다. 단일 애플리케이션 구조에서는 기능 하나를 수정할 때마다 전체를 재배포해야 한다. ■ 부하 확장성 — 검색은 호출 빈도가 높고, 영수증 인식은 빈도는 낮으나 자원 소모가 크다. 단일 구조에서는 가장 무거운 기능에 맞추어 전체를 증설해야 한다. ■ 장애 격리 — 특정 서비스의 장애가 다른 서비스로 전파되지 않는다. · 검증 결과 : 부하시험에서 레시피 검색만 포화하였으며, 해당 서비스만 4 배 수평 확장하여 해소하였다 (p95 2.7 초 → 46ms).
Kubernetes 채택 근거	MSA 는 확장 가능한 구조를 정의하는 단계이며, 실제 확장과 운영은 오케스트레이터가 담당한다. · 서비스별 독립 배포 · 무중단 전환 · 자동 복구 · 수평 확장을 선언형으로 확보한다. · NetworkPolicy · mTLS · RBAC 로 서비스 간 경계를 코드로 정의한다. · 동일 매니페스트를 온프레미스와 AWS 양측에 적용할 수 있어 이전 비용을 절감하였다.
온프레미스 선행 구축 근거	· 클라우드 비용 없이 Kubernetes 전 계층을 직접 구축·검증할 수 있다 (학생 예산 제약). · 매니지드 서비스가 추상화하는 계층(CNI · 스토리지 · 컨트롤플레인 · 인증서)을 직접 구성하였다. · 실장애 발생 이력 : 호스트 무흔적 급사 3 회 · 메모리 하드웨어 불량 · 디스크 데이터 오염 이를 반영하여 배치 원칙을 수립하였다 (quorum 다수는 호스트 B · primary 는 호스트 A).
AWS 이전 근거	① 가용성 — 물리 호스트 2 대는 랙·전원·회선을 공유하나, 2 개 AZ 는 이를 공유하지 않는다. ② 확장 상한 — 온프레미스는 노드 증설이 수작업이므로 트래픽 급증 대응에 수 시간이 소요된다. Karpenter 적용 시 노드 기동까지 51 초가 소요된다 (실측). ③ 관리 부담 이전 — etcd · 컨트롤플레인 · OS 패치 · 인증서를 AWS 책임 영역으로 이관한다. ④ DR — 온프레미스를 폐기하지 않고 Warm Standby 로 재활용하여 2 개 사이트 구성을 확보한다. ※ 매니지드 서비스를 일괄 채택하지는 않았다. RDS · OpenSearch · MSK 는 미채택이다. 해당 서비스의 주된 이점인 백업과 페일오버는 CloudNativePG · ECK 로 이미 확보된다.
DR 범위 설정 근거	재생성이 불가능한 데이터는 PostgreSQL 에 한정된다. · Elasticsearch — PostgreSQL 로부터 재색인 가능 (실측 1.0 초)

	· Redis — 캐시 용도이므로 빈 상태로 기동 가능 · Kafka — 드레인 후 재수집 · 따라서 WAL 을 S3 로 지속 아카이빙하여 온프레미스가 5 분 이내로 추종하도록 구성하였다 (RPO 5 분).
제약 조건	· 전 AI 기능 CPU 전용 — 보유 GPU(GTX 1060 3GB)로는 학습이 불가하여 CRF · XGBoost · LightGBM 으로 설계 (예외 : YouTube 영상 추출은 외부 Gemini API 사용 · 별도 승인) · 학생 예산 — GPU 인스턴스 미사용 · 셀프호스트 우선 · EC2 Spot 미채택 (비용이 아닌 구조적 사유) · 데이터 — 공공 오픈데이터 및 공식 API + 교육용 비상업 크롤링 (비공개 전제)

개발 일정 (마일스톤)

기간	단계	주요 산출물	비고
07.09 ~ 07.14	주제 선정 · 설계	도메인·데이터소스 확정 · 기술 스택 결정 · 아키텍처 설계	07-09 레포 생성
07.15 ~ 07.26	애플리케이션 개발	Front & Back 개발(12 종) · DB 설계 및 구축 · 데이터 파이프라인 · AI 모델(NER-RAG)	07-15 스키마 정본(SSOT) 확정
07.27 ~ 08.08	온프레미스 K8s 구축	kubeadm 5 노드 · Istio 서비스 메시 · 보안 설계(netpol-mTLS-RBAC) · CI/CD(Jenkins→ArgoCD) · 부하시험 · 카나리 배포	07-27 K8s 구축 착수(P0)
08.09 ~ 08.18	AWS Cloud 기반 EKS 이전	확정 결정 C-1~C-89 · Terraform/Ansible IaC · GitLab CI(arm64→ECR) · EKS-ALB-WAF · 데이터 이관 · Blue-Green · Karpenter · Lambda 10 종	08-09 AWS 이관 최우선 승격
08.19 ~	안정화 · 최종 발표 준비	이관 마무리 감사(F-1~F-10) · 관측 정합 · 시방서·명세서 · 발표 자료	—

산출물

설계 · 시방	· 통합 인프라 시방서 (Docker → 온프레임 K8s → AWS · 19 시트) · K8s 오브젝트 시방서 (온프레임 · 27 시트) / AWS Cloud 설계 시방서 및 비용 (19 시트) · 데이터 명세서 (전체 컬럼 323) · API 명세서 · AI 명세서 · 프론트 화면정의서 · WBS / 프로젝트 일정
문서 (레포 내)	· docs/ 마크다운 143 개 · 43,683 줄 — 영역별 정본(SSOT) 체계 · 확정 결정 C-1 ~ C-89 (AWS 이관 체크리스트) · ADR
코드	· 애플리케이션 12 종 (FastAPI · React) · 수집 파이프라인 · AI 모델 5 종 · Lambda 10 종 · IaC — Terraform (Proxmox · AWS) · Ansible · Kustomize 오버레이 2 별(onprem · eks)

	· 커밋 1,490 · PR/이슈 611 · 머지 523
운영 실증	· 무중단 Blue-Green 승격 1 초 · 롤백 2 초 · 전환 중 1,409 요청 5xx 0 건 · 부하 1,500 VU 에서 Karpenter 노드 2→4→6 자동 확장 · master 강제종료 3 분 3 초 동안 인그레스 중단 0 (151/151 HTTP 200) · 복구 26 초 · 데이터 이관 후 PostgreSQL 41 테이블 체크섬 일치 (crawl_raw 의도적 제외)

기대 효과

1. 서비스 측면	· 레시피 · 가격 · 예산으로 분리되어 있던 의사결정을 단일 흐름으로 통합한다. · 사용자는 식단과 소요 비용을 동일 화면에서 확인하고 결정할 수 있다. · 냉장고 재고와 영수증을 반영하여 실제 지출에 근접한 금액을 제공한다.
2. 데이터 자산	· 표준 품목 사전(약 300 종) · Gazetteer · 가격 이력이 누적될수록 매칭 정확도가 향상된다. · 가격 이력이 누적되면 현재 제외한 할인 주기 예측 기능을 재도입할 수 있다. · 대화 로그가 누적되면 의도분류 모델을 자체 데이터로 재학습할 수 있다.
3. 인프라 · 운영 측면	· 동일 매니페스트로 온프레미스와 AWS 를 모두 운영할 수 있어 특정 사이트에 종속되지 않는다. · 온프레미스를 폐기하지 않고 DR 대기 사이트로 재활용하여 2 개 사이트 구성을 저비용으로 확보하였다. · 배포·확장·복구가 선언형으로 정의되어 수작업 절차가 감소하였다. · 비용이 네임스페이스·워크로드 단위로 귀속되어(kubecost) 항목별 비용 파악이 가능하다.
4. 학습 측면	· 매니지드 서비스가 추상화하는 계층(CNI · 스토리지 · 컨트롤플레인 · 인증서)을 직접 구성·운영하였다. · 실장애(호스트 급사 · 메모리 불량 · 디스크 오염 · 캐시 스템피드 · 쿼터 병목)를 진단하고 그 결과를 설계 원칙과 문서로 정리하였다. · 동일 시스템을 Docker → Kubernetes → 매니지드 Kubernetes 로 두 차례 이전하며 매니지드 채택과 자체 운영의 판단 기준을 수립하였다.
5. 확장 가능성	· 서비스 추가가 기존 서비스에 영향을 주지 않는다 (12 종 → n 종). · 트래픽 급증 시 파드(HPA-KEDA)와 노드(Karpenter) 두 층이 자동으로 대응한다. · AI 트랙이 서버리스로 분리되어 있어 모델 교체가 클러스터 배포와 무관하다.